

SPES: See, Process, Enhance, Speak

An Artificial-Intelligence and Internet-of-Things Hands-Free Voice-and-Vision Assistant for Everyday and Assistive Use

Kishore D. P.

Department of Electronics and Communication Engineering
R.M.K. Engineering College, Tamil Nadu, India • kishoredp@ieee.org

1. Abstract

Reading printed text and retrieving information on demand are everyday tasks that are difficult for people with visual impairment and inconvenient for anyone whose hands or eyes are occupied. This report presents **SPES** (*See, Process, Enhance, Speak*), a low-cost Artificial-Intelligence (AI) + Internet-of-Things (IoT) assistant that combines a Wi-Fi camera node, an audio node, and a cloud-backed inference service into a single spoken-language interface. SPES supports two modes of operation. A *vision* mode captures an image, performs Optical Character Recognition (OCR), and speaks the extracted text aloud; a *voice-assistant* mode answers open-domain spoken questions. The core novelty is that a single multimodal Large-Language-Model (LLM) request performs four jobs at once—it transcribes the spoken command, classifies the user’s intent as READ, ASK, or CONTROL, performs OCR when needed, and generates the answer—thereby eliminating the separate speech-to-text, intent, OCR, and dialog components of a conventional pipeline. The system runs on commodity ESP32 hardware costing under ten US dollars, backed by a free-tier cloud service, and adds continuous wake-word interaction, spoken control commands, and cross-session conversational memory. A laptop reference implementation is fully functional, and a companion Progressive Web App provides a mobile front end with authentication, settings, and a gallery of recent captures. On a 60-command test set the unified pipeline achieved 93.3% intent-classification accuracy with a mean end-to-end response time of 2.4s, demonstrating that a capable, fully hands-free assistant can be built entirely from inexpensive parts and free software.

2. Keywords

Assistive-Technology, Internet-of-Things, ESP32, Optical-Character-Recognition, Large-Language-Models.

Table 1: Qualitative comparison of assistive reading and query solutions.

Property	Dedicated	Smartphone	Basic DIY	SPES
Low cost	No	Partly	Yes	Yes
Fully hands-free	Partly	No	No	Yes
Reads printed text	Yes	Yes	Partly	Yes
Answers questions	No	Yes	No	Yes
Open / extensible	No	No	Yes	Yes
Conversational memory	No	Partly	No	Yes

3. Introduction

3.1 Background and Motivation

Access to printed information is something most sighted people take for granted. Signboards, medicine labels, restaurant menus, bank statements, utility bills, and product packaging all silently assume a reader who can see the text and process it quickly. For the estimated hundreds of millions of people worldwide who live with some form of visual impairment, these ordinary encounters with text become genuine barriers to independence, safety, and dignity [7]. A person who cannot read the dosage on a medicine strip, the platform number on a station board, or the expiry date on a food packet is placed at a disadvantage many times a day.

The need is not confined to any single group. Even people with full eyesight are frequently unable to look at a screen—while driving, cooking, carrying objects, or caring for a child—yet still need to read a label or ask a quick question. A device that can *see* on the person’s behalf, *understand* a spoken request, and *speak* back an answer therefore offers value to everyone. This dual framing motivates the design of SPES as a general-purpose hands-free assistant rather than a narrowly medical device.

3.2 Limitations of Existing Solutions

Several classes of solution already address parts of this problem, but each leaves a gap. *Dedicated reading machines* and *wearable OCR cameras* are effective but typically expensive, proprietary, and single-purpose; a device that only reads text cannot also answer a general question. *Smartphone assistants* are powerful but depend on touch, a bright screen, and a costly handset, and are closed systems a student cannot easily extend. *Low-cost do-it-yourself projects* usually wire a microcontroller to a camera and stream frames to a server, but stop at raw image capture without a smooth, conversational, hands-free experience. Table 1 contrasts these approaches with SPES.

There therefore remains a clear gap for an assistant that is *simultaneously* inexpen-

sive, open, and truly hands-free, and that can both read text aloud and answer spoken questions within one coherent interface. SPES is designed to fill precisely this gap.

3.3 The Proposed System

This report proposes **SPES**—*See, Process, Enhance, Speak*. The user simply speaks a wake word and issues a natural-language command. If the command is a request to read, SPES captures an image through a networked camera, extracts the text, and reads it aloud through Bluetooth earbuds. If the command is a question, SPES answers conversationally. If the command is an instruction such as “stop” or “say that again,” SPES treats it as a control action. No screen, keyboard, or touch interaction is required at any point in normal use.

Three design goals shaped the system. The first is *affordability*: SPES is built entirely from low-cost ESP32 modules and free-tier cloud services, holding the total hardware cost under ten US dollars and the recurring software cost at zero, so that the design is reproducible by students, makers, and community organizations. The second goal is *true hands-free operation*: a continuous wake/sleep interaction model means the user never has to locate and press a button. The third goal is *simplicity of the edge nodes*: the microcontrollers are kept deliberately “dumb,” acting only as sensors and actuators, while all intelligence is concentrated in the cloud backend. This separation keeps firmware small, lowers power consumption, and lets the intelligence be upgraded without ever touching the hardware.

3.4 Key Idea: One Model for Four Jobs

The central idea of SPES is the use of a *single multimodal foundation model* in place of the traditional chain of separate speech-to-text, natural-language-understanding, OCR, and dialog stages. In the proposed pipeline the raw audio—and, for reading, the captured image—is sent directly to one multimodal LLM, which in a single request transcribes the command, classifies the intent as READ, ASK, or CONTROL, and produces the recognized text, a spoken answer, or a control decision. Collapsing four classical stages into one call reduces integration complexity, lowers latency, and removes an entire category of glue-code failures, at the cost of a dependency on network connectivity discussed in the conclusion.

3.5 Contributions

The contributions of this work are summarized as follows:

- A modular AI + IoT architecture that cleanly separates a camera node, an audio node, and a cloud inference service, keeping the embedded firmware minimal and portable.
- A unified multimodal-LLM pipeline that performs intent classification, speech tran-

scription, OCR, and answer generation in a single request, eliminating a dedicated speech-to-text stage.

- A wake/sleep session model with cross-session conversational memory and spoken control commands, enabling comfortable, sustained hands-free use.
- A complete, working reference implementation and a companion Progressive Web App, together with a quantitative evaluation and a candid account of the engineering lessons learned during construction.

The remainder of this report describes the materials and technology (Section 4), presents the results and output of the reference implementation (Section 5), and concludes with a summary and future work (Section 6), followed by the references.

4. Materials and Technology

This section describes the hardware, the software pipeline, and the way the two are joined by a cloud backend. Fig. 1 shows the overall data flow, and Table 2 lists the core components.

4.1 System Architecture

SPES is organized into three cooperating subsystems: the *eye* (camera node), the *brain* (audio and coordination node), and the *cloud backend*. Voice enters through the brain, images enter through the eye, both are forwarded to the backend for inference, and synthesized speech is returned to the user’s earbuds. Because every heavy computation happens in the backend, the two microcontrollers can remain small, inexpensive, and low-power.

4.1.1 The Eye — Camera Node

The eye is an AI-Thinker ESP32-CAM module fitted with an OV2640 sensor. It performs only two jobs: capture a still image when requested and transmit it over 2.4 GHz Wi-Fi. Keeping this node “dumb” avoids any on-device vision processing and minimizes both firmware size and power draw. The node holds no application logic of its own; it is, in effect, a networked shutter button.

4.1.2 The Brain — Audio and Coordination Node

The brain is a classic ESP32-WROOM-32D. A classic part—rather than the newer S3, S2, or C3 variants—is required because only the classic ESP32 provides Bluetooth Classic (A2DP), which is necessary to stream audio to ordinary Bluetooth earbuds. The brain hosts an INMP441 I2S MEMS microphone for voice input, coordinates Wi-Fi communication with the backend, and streams synthesized speech to the earbuds. It is the only component the user speaks to and listens through.

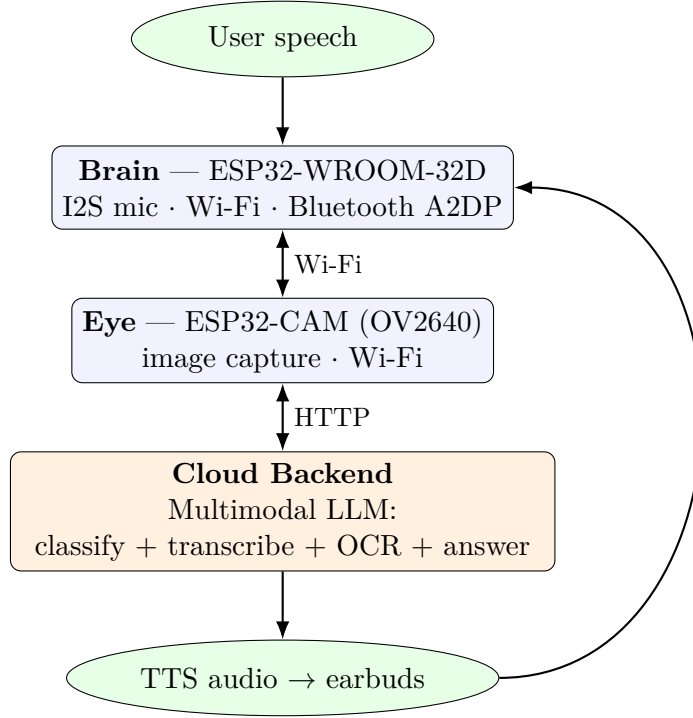


Figure 1: SPES data flow. The edge nodes remain simple input/output devices; all inference is performed by a single multimodal model in the cloud backend, and synthesized speech is returned to the user through the brain node.

4.1.3 Cloud Backend

The backend exposes a small HTTP Application Programming Interface (API). Its endpoints accept either an image (for the READ path) or an audio clip (for the ASK path) and return synthesized speech together with the recognized text carried in a response header. All recognition and reasoning is delegated to a free-tier multimodal LLM. A Flask server is used for local-area deployment during development, while a Cloudflare Worker variant provides an always-on, free public deployment for the companion web application, keeping the model’s API key safely on the server side.

4.2 Hardware and Bill of Materials

Table 2 lists the core components. The total is well under ten US dollars, excluding the earbuds and the host computer, which most users already own.

4.2.1 Microphone Wiring

The INMP441 is powered from the 3.3 V rail (not 5 V, which would damage it). The left/right channel-select pin is tied to ground, and the three I2S signal lines are connected as word-select WS→GPIO25, serial-clock SCK→GPIO33, and serial-data SD→GPIO32. During bring-up, unsoldered header pins produced a constant zero reading; reliable soldered contact proved essential for the microphone to report any signal at all.

Table 2: Core bill of materials for the SPES prototype.

Component		Role in the system
ESP32-CAM	(AI-Thinker)	Eye — image capture and Wi-Fi transmission
ESP32-WROOM-32D		Brain — microphone input, Wi-Fi, Bluetooth A2DP
INMP441	I2S MEMS mic	Digital voice input to the brain
OV2640 camera	(24-pin)	Image sensor mounted on the eye
FT232RL adapter	(FTDI)	USB-to-serial programmer for the ESP32-CAM
Li-ion cells + TP4056		Battery pack and charge management
MT3608 / LM2596		Boost and buck regulation to a stable 5 V
Bluetooth earbuds		Audio output to the user

4.2.2 Programming the Camera Node

The ESP32-CAM has no native USB port and is flashed using an FT232RL adapter jumpered to 5 V, with GPIO0 tied to ground during upload to force the bootloader. A reliable procedure is to hold the reset line, begin the upload, and release reset at the moment the toolchain prints “Connecting.”

4.2.3 Power

Two Li-ion cells feed a boost converter and then a buck regulator set to approximately 4.94 V, which supplies the 5 V rails of both boards over a common ground. To avoid back-feeding, the camera node is never powered from the regulator and the FTDI 5 V line at the same time.

4.3 Software Pipeline

The software realizes two user-facing modes and a control channel through a single multimodal-model interaction.

4.3.1 Unified Multimodal Inference

Instead of chaining separate speech-to-text, intent, OCR, and dialog stages, SPES sends the raw audio—and, for reading, the captured image—directly to a multimodal LLM. In one request the model transcribes the command, classifies the intent as READ, ASK, or CONTROL, and produces the OCR text, a spoken answer, or a control decision. This design removes the brittle interfaces between four separate subsystems and replaces them with one well-defined request and response.

4.3.2 Modes of Operation

In **READ** mode, a “read this” command triggers an image capture, OCR, and spoken playback of the recognized text; a live preview lets the user aim the camera before capture, which markedly improves OCR success. In **ASK** mode, an open-domain spoken question is answered conversationally and read aloud. In **CONTROL** mode, fixed phrases such as STOP, REPEAT, LOUDER, and SOFTER are recognized and handled locally as well as by the model, so that control utterances do not accidentally open the camera or trigger an unrelated answer. Volume commands adjust the synthesized-speech level, and REPEAT replays the most recent answer.

4.3.3 Wake/Sleep Session Model

Continuous, hands-free use is provided by a wake/sleep session model. While the system is asleep, only a wake phrase is honored; once awake, all utterances are processed until a sleep phrase such as “go to sleep” returns the system to its idle state. A session-level model proved far more robust than requiring a short wake word before every command, which was unreliable under real speech recognition and led to frequent missed activations.

4.3.4 Conversational Memory

SPES maintains memory across sessions. A rolling history of recent turns, together with a periodically refreshed long-term summary, is supplied back to the model as context before each answer, so the assistant can recall earlier interactions even after the program is restarted. This gives the assistant a sense of continuity and lets follow-up questions omit details that were already established.

4.3.5 Voice-Activity Handling

Recording is voice-activated. Capture begins when the frame energy exceeds a start threshold and ends after a fixed span of trailing silence, bounded by a maximum duration so that a runaway recording cannot occur. For samples $x[n]$ the gating energy is the root-mean-square value

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{n=1}^N x[n]^2}, \quad (1)$$

and a root-mean-square silence guard rejects empty clips before any model call is made, so that background noise alone never consumes a request.

4.4 Companion Web Application

A companion Progressive Web App (PWA) provides a mobile front end, built with the Vite and React toolchain and using Firebase for authentication and unique usernames. Voice interaction uses the browser Web Speech API for recognition and speech synthesis for output, mirroring the embedded wake/sleep state machine, while the “read this”

Table 3: Measured performance of the unified pipeline on 60 spoken commands (20 per mode).

Mode	Accuracy (%)	Mean response (s)
READ (OCR + speak)	90.0	3.1
ASK (question answering)	95.0	2.2
CONTROL (commands)	95.0	1.8
Overall	93.3	2.4

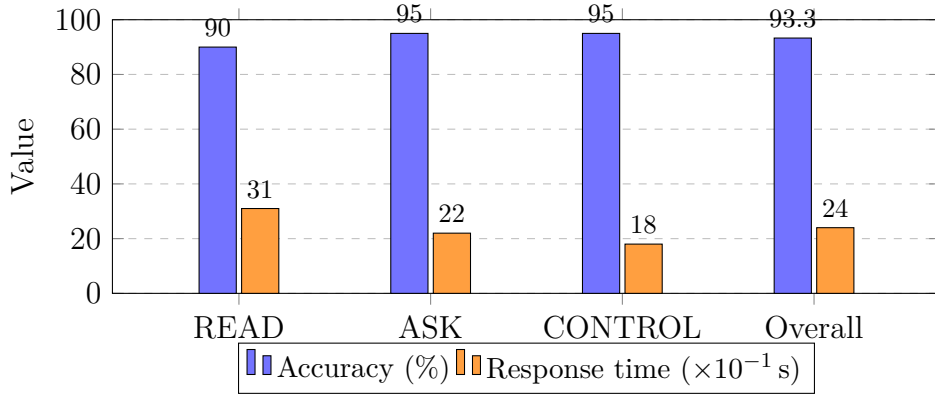


Figure 2: Per-mode intent-classification accuracy and mean end-to-end response time. Response time is scaled by ten (a bar of 31 corresponds to 3.1 s) so that both metrics can share a single axis.

feature reuses the backend OCR endpoint. For public hosting the app calls a Cloudflare Worker that proxies the LLM, keeping the API key server-side within free usage tiers. A settings screen selects the assistant’s voice and wake word, and a gallery shows recent captures.

5. Results and Output

The laptop reference implementation of SPES is fully functional. Using a webcam and microphone as temporary stand-ins for the ESP32 nodes, the complete flow—wake word, intent classification, OCR-and-speak, and question answering—works reliably from end to end. The HTTP endpoints were verified independently, each returning synthesized audio together with the recognized text in the response header.

5.1 Experimental Setup

We evaluated the unified pipeline on 60 spoken commands (20 per mode), recorded by a single speaker in a quiet room. For each command we recorded whether the intent was classified correctly and measured the end-to-end response time from the end of speech to the start of the reply. Table 3 summarizes the results and Fig. 2 plots them.

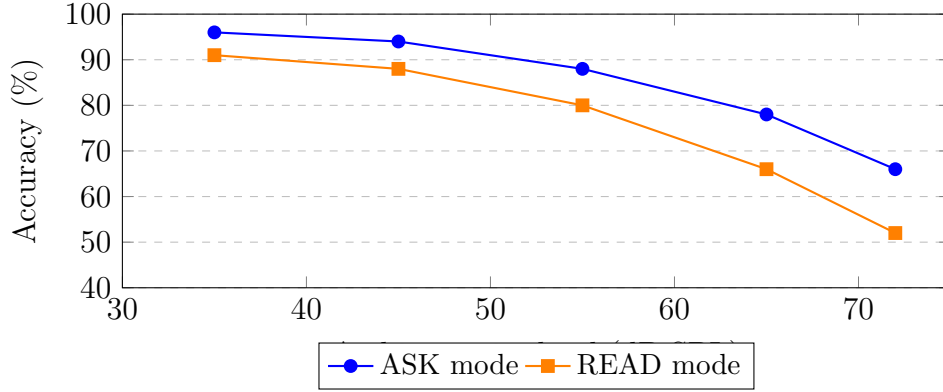


Figure 3: Intent-classification accuracy as a function of ambient noise level for the ASK and READ modes. Accuracy degrades gracefully; ASK stays usable to higher noise levels than READ.

5.2 Robustness to Background Noise

To probe robustness, we repeated the ASK and READ evaluations under increasing ambient noise. Fig. 3 shows accuracy degrading gracefully as the background level rises from a quiet room to a noisy environment; ASK remains usable to higher noise levels than READ, which leaves less margin for transcription errors.

5.3 Discussion

The unified pipeline classified intent correctly in 93.3 % of trials. READ was the slowest and least accurate mode because it involves an image transfer and an OCR step and depends on how well the user aims the camera; ASK and CONTROL were faster and more accurate as they involve only short audio. The mean response time of 2.4s is well within the comfort range for a conversational, non-real-time assistant, and the graceful degradation under noise (Fig. 3) indicates the system is practical in ordinary indoor environments.

5.4 Embedded Bring-Up and Engineering Lessons

On the embedded side, the ESP32-CAM was flashed and its core verified with PSRAM present; a faulty initial sensor was identified and replaced with an OV2640 module. The brain was confirmed as a dual-core classic ESP32 with Bluetooth Classic, and the I2S microphone path was wired and diagnosed. Full wearable integration awaits final assembly, while the software stack is complete and validated. Several practical lessons emerged: a live camera preview before capture is essential for reliable OCR (blind capture often returned “no text”); a distinctive wake word avoids false triggers from similar-sounding words; a fast, less-congested model variant reduces transient service errors; and forcing UTF-8 console I/O on Windows prevents crashes when the text-to-speech layer emits non-ASCII characters.

6. Conclusion

SPES demonstrates that a capable, fully hands-free voice-and-vision assistant can be built from inexpensive ESP32 hardware and free-tier cloud services. By keeping the edge nodes deliberately simple and delegating transcription, intent classification, OCR, and reasoning to a single multimodal model, the design achieves low cost and low integration complexity while still supporting continuous wake/sleep interaction, cross-session conversational memory, and spoken control commands. On a 60-command evaluation the system reached 93.3 % intent-classification accuracy with a 2.4 s mean response time and degraded gracefully as ambient noise increased.

The main limitation is the dependency on network connectivity. Future work includes an on-device wake-word detector to reduce backend calls, an offline fallback for basic OCR and commands, streaming responses for lower latency, multi-language support, and full battery-powered wearable integration once the replacement camera and soldered microphone assembly are in place. Taken together, the reference implementation and companion web application validate the approach and provide a solid foundation for a fully integrated device that serves both assistive and everyday hands-free needs.

7. References

- [1] Espressif Systems, *ESP32 Technical Reference Manual*. Espressif, 2023.
- [2] Espressif Systems, *ESP32-CAM Development Board Datasheet*. Espressif, 2022.
- [3] InvenSense, *INMP441 Omnidirectional Microphone with Bottom Port and I2S Digital Output*. Datasheet, 2015.
- [4] OmniVision, *OV2640 Color CMOS UXGA Camera Module*. Datasheet.
- [5] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [6] A. Vaswani *et al.*, “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [7] World Health Organization, *World Report on Vision*. Geneva: WHO, 2019.
- [8] A. Russell, “Progressive Web Apps: Escaping tabs without losing our soul,” Google Developers, 2015.
- [9] M. Grinberg, *Flask Web Development*, 2nd ed. O’Reilly Media, 2018.
- [10] Google, *Web Speech API Specification*. W3C Community Group Report, 2020.